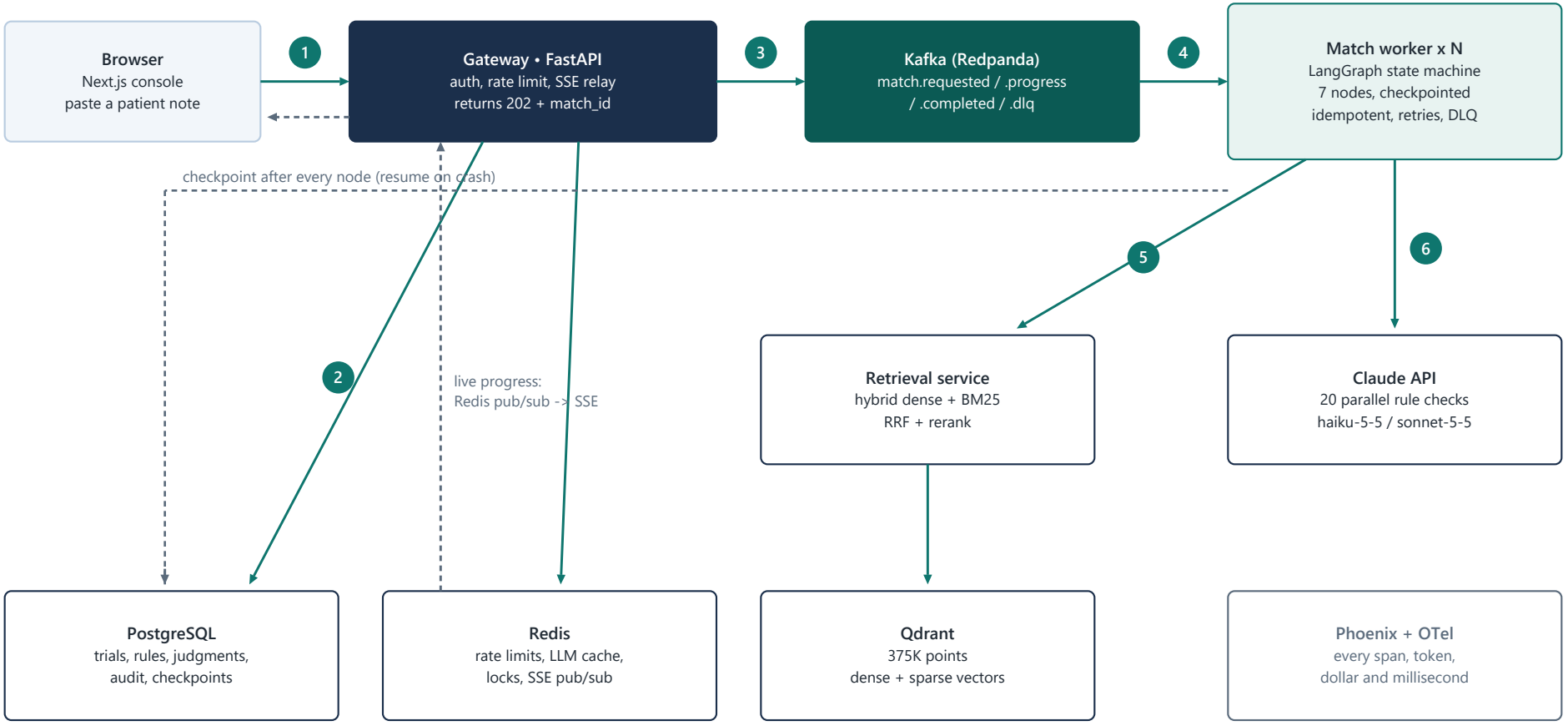
1 • End-to-end system

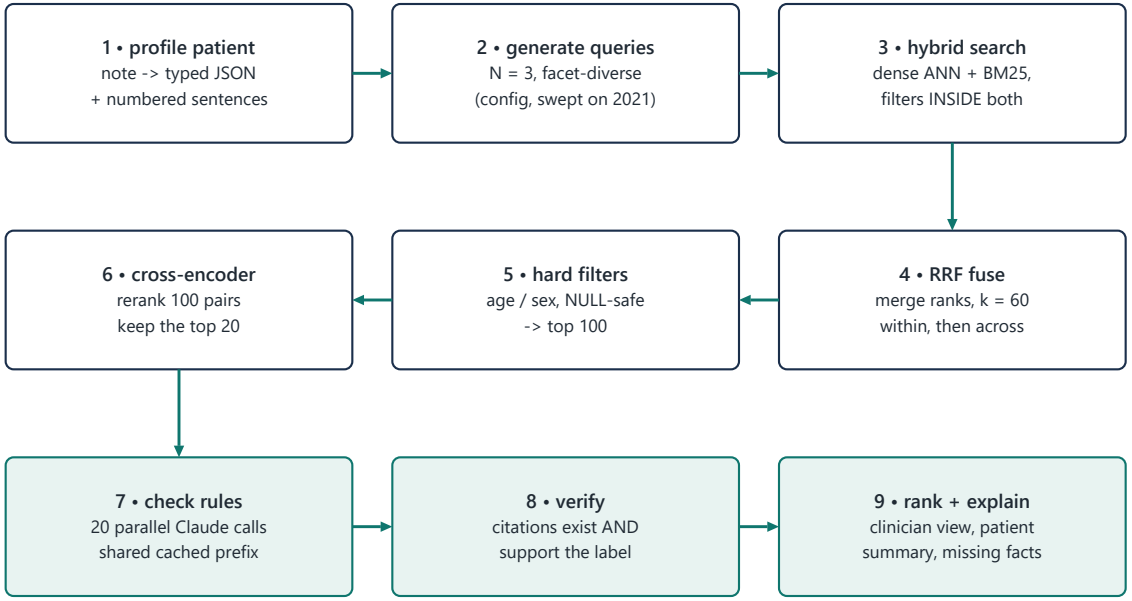
One patient note, from the browser to a ranked answer. Solid arrows are the request path; dashed arrows are checkpoints and live progress.



1 POST /v1/matches with the patient note. Gateway checks API key + rate limit (Redis).
2 Request row AND the outbox event are written to Postgres in ONE transaction.
3 match.requested published from the outbox, keyed by match_id so events stay ordered.
4 A worker in the consumer group picks it up; offsets commit only after a checkpoint.
5 Worker calls the retrieval service: 375,000 -> 100 -> 20 candidate trials.
6 Worker fans out 20 parallel Claude calls, one per trial, sharing a cached prompt prefix.
7 Results to Postgres, match.completed published, progress streamed live throughout.

2 • The RAG pipeline

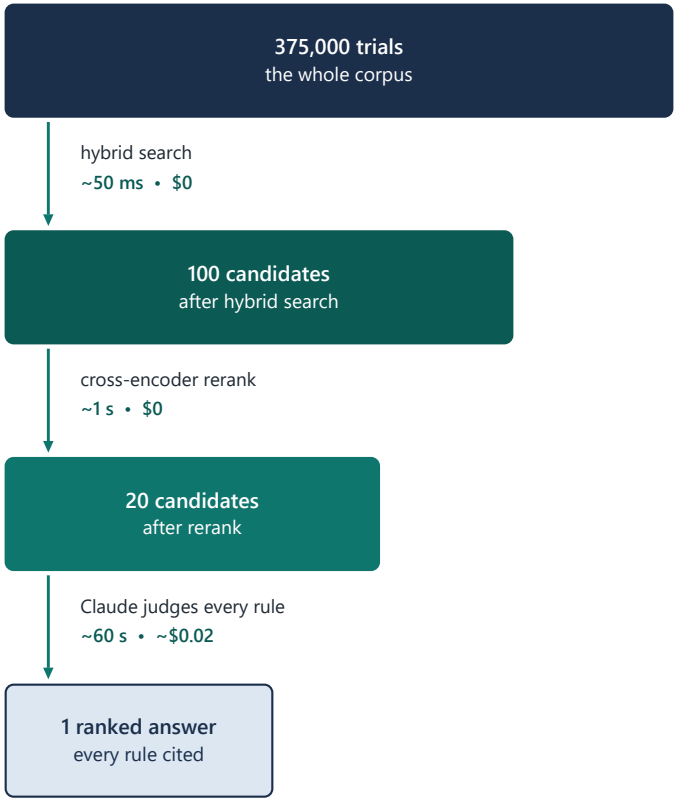
Left: the nine steps, reading left to right then wrapping. Right: what each stage does to the candidate count, and what it costs.



Steps 1-6 are retrieval: no LLM reads a trial, and nothing costs money.
Steps 7-9 (green) are the read stage: ~95% of all token spend lives in step 7.

The filter is NOT a stage after step 3 - it is a constraint evaluated inside both searches.

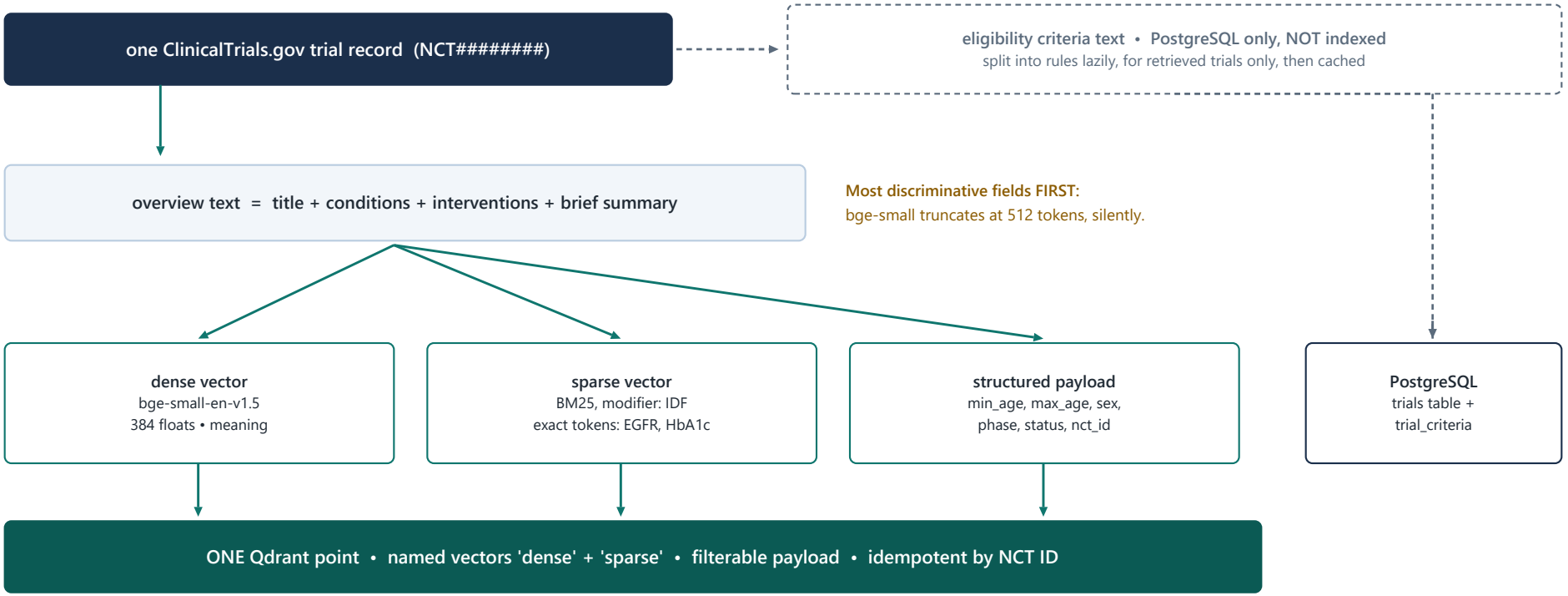
Cache breakpoint sits after the numbered patient note, so 19 of 20 calls read it back at 1/10 price.



Recall@100 governs the whole system: a trial dropped at the first stage can never be recovered later.

3 • Ingestion (offline, once)

How 375,000 trials become a searchable index. Runs on Day 2 and then only when the corpus changes.



375,000 x 384 dims x 4 bytes = about 576 MB of dense vectors, plus a small sparse index.
One CPU pass, roughly 30-90 minutes. Runs overnight. Re-running it changes nothing.

Dual write: Postgres first, then Qdrant. The ingest job must be resumable and idempotent by NCT ID - this is the price paid for choosing Qdrant over pgvector, and the only place the two can drift apart.

4 • Toolkit

Every moving part, what it is, and how deeply you need to know it. Depth is what an interviewer will probe, not what the README lists.

Tool	What it is, in plain words	Its job in Beacon	Runs	Port	Cost	Depth
Python 3.12 + uv	the language; uv manages the monorepo and locks deps	everything backend	local	-	$0	**Deep**
FastAPI	framework for building web APIs in Python	gateway + retrieval service	local	8000 / 8001	$0	**Deep**
Claude API	the LLM, over HTTP	profiling, queries, rule checks, summaries, judge	cloud	-	**~$20-40 total**	**Deep**
LangGraph	multi-step AI workflows as a graph with saved state	the match pipeline: 7 nodes, checkpoints, fan-out	local	-	$0	**Deep**
Qdrant	vector database for dense + sparse vectors with filters	hybrid search over 375K trials	docker	6333	$0	**Deep**
FastEmbed	runs small embedding models locally via ONNX	bge-small (dense) and bm25 (sparse)	local CPU	-	$0	**Deep**
`bge-reranker-base`	cross-encoder: reads query+trial together, scores the pair	reranks the top 100 down to 20	local CPU	-	$0	**Deep**
PostgreSQL	the standard relational database	source of truth, audit log, LangGraph checkpoints	docker	5432	$0	Medium
Redis	very fast in-memory store	rate limits, LLM cache, locks, SSE pub/sub	docker	6379	$0	Medium
Kafka (Redpanda)	a durable log of messages services publish and read	job queue, progress events, retries, DLQ	docker	9092	$0	Medium
OTel + Phoenix	tracing standard, plus a UI built for LLM apps	one trace per request: every span, token, dollar	docker	6006	$0	Medium
Docker Compose	runs every service with one command	local deployment, 5 containers	local	-	$0	Medium
ranx	IR evaluation library	nDCG@10, Recall@100, fusion, significance tests	local	-	$0	Medium
ir_datasets	loader for standard IR benchmarks	TREC corpus, topics and qrels	local	-	$0	Medium
GitHub Actions	CI that runs on every push	lint, types, tests, the evaluation gate	cloud	-	$0	Medium
Next.js	React web framework	thin demo console, one or two pages	local	3000	$0	Light
MCP server	lets AI assistants call your tools	exposes search_trials / check_eligibility (phase 2)	local	-	$0	Light

5 • Contracts

Fixed in Wave 0 and read-only to every agent afterwards. If one of these has to change, the orchestrator changes it once, everywhere.

PostgreSQL table	Holds
`trials`	NCT ID, title, conditions, interventions, phase, status, min/max age, sex, locations, raw eligibility text
`trial_criteria`	one row per rule: trial ID, inclusion/exclusion, position, text. Parsed once, reused for every patient
`patients`	synthetic notes, split into numbered sentences, plus the extracted profile (JSON)
`match_runs`	one row per request: status, timings, total tokens, total cost, config version
`trial_matches`	per patient-trial pair: retrieval rank, final score, eligibility category, summary
`criterion_judgments`	per rule: label, cited sentence IDs, explanation, verifier verdict
`llm_calls`	audit log: model, prompt hash, input/output/cached tokens, cost, latency, billed tier, request ID
`feedback`	reviewer corrections, which become future evaluation data
`eval_runs`	benchmark results per configuration, so runs can be compared over time
`outbox`	events waiting to be published to Kafka (transactional outbox pattern)

Kafka topic	Purpose	Redis key	Purpose
`match.requested`	new jobs; key = match_id so one job's events stay ordered	`rl:{api_key}`	token-bucket rate limiter, atomic Lua
`match.progress`	step-by-step updates for the live console	`idem:{key}`	idempotency, so a retried POST starts no second job
`match.completed`	finished results, for evaluation and analytics	`llm:{sha256}`	exact-match cache of LLM responses
`match.retry` / `match.dlq`	delayed retry; after N failures, parked for inspection	`lock:ingest`	only one ingestion job runs at a time
`trials.updated`	phase 2: daily changed trials, re-indexed incrementally	`chan:match:{id}`	pub/sub feeding SSE to the right gateway replica

```
# libs/beacon-core/src/beacon_core/schemas.py  -  owned by Wave 0, read-only to every agent

class CriterionJudgment(BaseModel):
    criterion_id: str
    kind:  Literal['inclusion', 'exclusion']
    label: Literal['met', 'not_met', 'insufficient_info', 'not_applicable']
    evidence_sentence_ids: list[int]   # must point at real sentences in the numbered note
    explanation: str                   # one or two sentences, for the clinician view

PatientProfile | NumberedNote | TrialCandidate | MatchState  -  same file, same rule
```

6 • Scaling: what breaks first

In bottleneck order. Be able to say this out loud - 'we chose Qdrant to scale' is the wrong answer, because 375,000 vectors is small for any engine.

#	Bottleneck	When it bites	Why	The fix
1	**Cross-encoder on CPU**	first, under any real concurrency	~100 query-trial pairs per second, and it cannot be precomputed	GPU, ONNX int8 quantization, batching, or cache (query, trial) scores
2	**LLM cost and latency**	immediately, commercially	linear in patients x trials; 20 calls per patient at ~60 s	prune harder before step 7, prompt caching, Batches API, cheaper model per node
3	**Qdrant single node**	around 10M+ vectors	375K is small; HNSW wants to sit in RAM	shard the collection, add replicas
4	**Postgres single primary**	when analytics compete with writes	the llm_calls audit log grows fast and gets queried hard	read replicas; move audit analytics off the primary
5	**Rule parsing on the hot path**	high trial churn	parsed lazily per retrieved trial, cached - fine now, not forever	precompute all rules offline (this is the phase-2 rule-level index anyway)

Phase 2, ordered by what actually raises the ceiling. 1. Reviewer feedback loop, so corrections grow the evaluation set automatically - the one genuinely product-shaped feature, because it turns usage into an asset. 2. Live ClinicalTrials.gov sync: recruiting-only, distance to site - what makes it demoable to someone outside academia. 3. MCP server exposing search_trials / check_eligibility. 4. Cost-per-patient dashboard. 5. MedCPT embeddings and rule-level contextual chunks, gated on ablations. 6. Fairness slices and a prompt-version A/B harness. 7. A documented FHIR/EHR integration path - the answer to 'what would a hospital need?'